

Project Proposal: SlingShot

One-Sentence Pitch: An offensive-security agent that probes APIs for logic flaws like BOLA (Broken Object Level Authorization), then automatically writes defensive middleware to block the exploits it discovers.

Target User: Backend Engineers and AppSec Teams who want to test their APIs against "AI-speed" adversarial logic before deployment.

The AI's Role:

The AI acts as a **Dual-Agent System**. The first agent (The Attacker) "reads" an OpenAPI spec and brainstorms non-obvious business logic attacks (e.g., "Can I access a private invoice by changing the ID in the URL while authenticated as a different user?"). The second agent (The Fixer) analyzes the successful attack traces and generates a specialized **Security Middleware** (in Node.js/Python) to sanitize inputs and enforce stricter object-level checks.

Why a single extraction call isn't enough:

A single call cannot perform **iterative fuzzing**. To find a logic flaw like BOLA, the AI must first map the API's behavior, then attempt a "baseline" request, then attempt a "mutated" request (the attack), and finally evaluate the status code and response body to confirm the breach. This requires a **multi-turn feedback loop** where the AI adjusts its "payload" based on the server's response. Furthermore, generating a "Fix" requires a separate reasoning step that synthesizes the vulnerability data with the existing codebase architecture.

Evaluation Metric:

$$\text{Exploit-to-Patch Success Rate} = \frac{\text{Verified Vulnerabilities with a Valid Fix}}{\text{Total Vulnerabilities Discovered}}$$

(A "Valid Code Fix" is defined as a patch that, once applied, causes the same attack payload to return a 403 Forbidden without breaking core functionality.)

Test Cases

Test Case 1: BOLA (Broken Object Level Authorization)

- **Input:**
 - *API Spec:* `GET /api/v1/invoices/{invoice_id}`
 - *Scenario:* User A tries to access `invoice_id: 999` (owned by User B).
- **Passing Output:**
 - *Red Phase:* "Exploit Confirmed: Accessing `/invoices/999` as User A returned a 200 OK with User B's data."
 - *Blue Phase:* "Fix Generated: Added an ownership check middleware to verify `request.user.id === invoice.owner_id` before returning data."

- **Failing Output:** "No vulnerability found" or "Fix generated: Add an API key" (Fails to address the *logic* flaw of ownership).

Test Case 2: Mass Assignment (Security Misconfiguration)

- **Input:**
 - *API Spec:* `PATCH /api/v1/user/profile` (Fields: `bio`, `avatar`).
 - *Scenario:* Attacker tries to include `"is_admin": true` in the JSON body.
- **Passing Output:**
 - *Red Phase:* "Exploit Confirmed: Server accepted 'is_admin' field and updated user permissions."
 - *Blue Phase:* "Fix Generated: Implemented an allow-list for the PATCH endpoint to only permit 'bio' and 'avatar' fields."
- **Failing Output:** "Error: invalid JSON" or "Fix: use HTTPS."

The 7-Day Plan (SlingShot)

Day	Focus	Deliverable
1	Spec Ingestion	A parser that reads an OpenAPI (Swagger) JSON and identifies "High Value" endpoints (GET/PATCH/DELETE).
2	The Attacker Agent	Build the OpenAI prompt loop that generates "Adversarial Intent" (e.g., "Try to read another user's data").
3	Live Fuzzing	Implement the Tool-Calling loop where the AI makes real <code>fetch</code> calls to a local "Victim API" and logs responses.
4	Vulnerability Judge	An LLM-based "Judge" that reviews the logs to decide if the attack was successful (e.g., looking for leaked PII).
5	The Fixer Agent	The AI analyzes the successful exploit and generates a Middleware Patch (Express.js or FastAPI).

6	UI: The Duel View	A frontend showing the "Attacker's Terminal" on the left and the "Proposed Security Patch" on the right.
7	Documentation	A README.md with a "Victim API" included so users can run their first SlingShot audit in 5 minutes.

Riskiest Component:

The **Exploit Verification Logic** is the most difficult part; the AI must be able to distinguish between a "Success" (a real vulnerability) and a "False Positive" (like a 200 OK that just contains an error message). If the AI can't accurately judge its own attacks, it will generate useless "fixes."